

WEB TABLETOP PLATFORM

Firm-Ready Product, Architecture, Delivery and Procurement Specification

Browser-native tabletop creation, publishing and multiplayer runtime

Implementation-ready for Phase A technical validation and vendor solicitation

Authoritative consolidated specification, including the finished UX/DX contract and screen-by-screen product behavior.
Superseded exploratory ideas are intentionally omitted.

Document Control

Item	Value
Package status	Implementation-ready for Phase A vendor solicitation and technical validation
Primary audience	Development firms, technical leads, product designers, QA, security, procurement
Product category	Browser-native 3D tabletop runtime + UGC creator platform
Core technology baseline	React, Babylon.js, Bun, Lua/Wasmoon baseline, Cloudflare Workers/Durable Objects, D1, R2, WebSocket, WebRTC transient
Authority	This v2 package supersedes the prior PDF and all earlier conversational exploration.
Revision	v2 - includes SPEC 10/11 finished UX/DX and latest decision updates

Contents

- SPEC 00 - Executive Brief
- SPEC 01 - Product Requirements
- SPEC 02 - UX & Interaction Specification
- SPEC 03 - Deterministic Game Kernel
- SPEC 04 - Lua API v1
- SPEC 05 - Multiplayer Protocol & Room Service
- SPEC 06 - Editor & Creator System
- SPEC 07 - Platform, Data, Security & UGC
- SPEC 08 - Acceptance, Delivery & Traceability
- SPEC 09 - Statement of Work & Vendor Procurement Requirements
- SPEC 10 - Finished UX & Creator DX Contract
- SPEC 11 - Screen-by-Screen UX & DX Contract
- Appendix A - Consolidated Decision Register
- Appendix B - Component Registry v1
- Appendix C - Canonical Action Registry v1
- Appendix D - Reference Schemas, Glossary & Traceability
- Appendix E - Architecture Diagrams
- Appendix F - Vendor Response Template
- Appendix G - Vendor Pricing Template
- Appendix H - Risk Register & Phase A Decision Gates
- Appendix I - Vendor Scoring Matrix
- Appendix J - Phase A Acceptance Worksheet
- Appendix K - Bid Comparison Worksheet

SPEC 00 - Executive Brief

Status: Normative

Purpose: Define the product, governing architectural thesis, non-negotiable principles and phased delivery model.

00.1 Product summary

Build a browser-native multiplayer tabletop platform with an integrated creator environment. Users create tabletop games visually and with Lua, playtest locally or online, publish immutable releases, browse community games, host live tables, invite friends, and play directly in the browser.

CREATE -> PLAYTEST -> PUBLISH -> BROWSE -> HOST -> INVITE -> PLAY

The intended product category is a web-native tabletop game runtime and UGC platform, not a single board game and not a general-purpose realtime video-game engine.

00.2 Core user promise

PRINCIPLE-001 - Browser native - MUST

A player receiving a valid invite link can join and play without installing native software.

PRINCIPLE-002 - Low-friction joining - MUST

An unauthenticated guest can join an invite-only table without creating an account. Target flow: invite link -> display name -> load -> table.

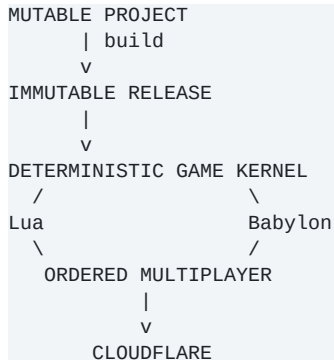
PRINCIPLE-003 - Immediate interaction - MUST

Normal local tabletop manipulation visually responds without waiting for network acknowledgement.

PRINCIPLE-004 - Creator accessibility - MUST

A creator can assemble a basic tabletop game without Lua; Lua adds behavior and rules.

00.3 Architectural thesis



00.4 Technology baseline

Concern	Baseline
Application/UI	React
3D rendering	Babylon.js
Tooling/monorepo	Bun
Creator scripting	Lua
Lua implementation	Wasmoon baseline
Code editor	CodeMirror 6
Editor docking	React FlexLayout
Hosting/backend	Cloudflare Workers
Live room coordinator	Durable Objects
Relational metadata	D1 baseline
Blob/assets	R2 baseline
Canonical transport	WebSocket
Transient transport	WebRTC when available

00.5 Locked architectural requirements

ARCH-001 - Deterministic kernel - MUST

Canonical gameplay executes through a deterministic Game Kernel.

ARCH-002 - Rendering separation - MUST

Babylon.js is presentation/rendering infrastructure, not canonical authority.

ARCH-003 - Host independence - MUST

Original room Host is not simulation authority; host disconnection does not end the table.

ARCH-004 - Ordered canonical actions - MUST

Canonical gameplay is reconstructable as starting snapshot + ordered actions.

ARCH-005 - Rollback/replay - MUST

Clients support rewind/replay enough to reconcile prediction and late canonical ordering.

ARCH-006 - Room coordinator - MUST

Each active table has one logical Durable Object room coordinator under the baseline architecture.

ARCH-007 - Backend does not simulate game rules - MUST

The Room DO sequences and coordinates; it does not run ordinary game-rule simulation or Babylon physics.

ARCH-008 - Canonical WebSocket path - MUST

Canonical gameplay remains operational through WebSocket when WebRTC is unavailable.

ARCH-009 - WebRTC optional - MUST

WebRTC is transient/presentation optimization only.

ARCH-010 - Deterministic randomness - MUST

Canonical randomness uses a versioned deterministic PRNG included in canonical state.

ARCH-011 - Immutable releases - MUST

Published playable releases are immutable.

ARCH-012 - Compatibility - MUST

Published releases pin compatibility information required to retain gameplay semantics.

ARCH-013 - Lua isolation - MUST

Creator Lua has no unrestricted JS/DOM/network/storage/filesystem/OS access.

ARCH-014 - Persistent Lua state - MUST

Persistent gameplay state controlled by Lua is Kernel-managed serializable state.

ARCH-015 - Playtest isolation - MUST

Playtest mutations never modify authored project state.

ARCH-016 - Presentation physics - MUST

Cross-browser deterministic rigid-body physics is out of scope; physics may be local presentation.

00.6 Security model

SEC-001 - UI-private information - MUST

The official client respects private hands, face-down cards, hidden containers and logical visibility.

SEC-002 - No hostile-client secrecy guarantee - MUST

MVP does not claim secrecy against a modified participant client. Do not market it as cryptographically secure competitive hidden-information infrastructure.

SEC-003 - Backend authorization remains trusted - MUST

Friendly multiplayer semantics do not weaken authorization for projects, publishing, room administration, saves or moderation.

00.7 MVP exclusions

- Voice/video chat
- VR/AR
- Native desktop/mobile apps
- Mobile creator editor
- Paid games/marketplace/creator payouts
- Visual scripting

WEB TABLETOP PLATFORM - FIRM HANDOFF PACKAGE

- Collaborative multi-user editor
- General-purpose realtime action games
- Deterministic rigid-body physics
- Ranked matchmaking
- Cryptographically secure hidden-state mode
- Steam Workshop integration
- Advanced social network

00.8 Delivery phases

Phase A - Technical Validation
Phase B - Core Runtime
Phase C - Creator + Local Play
Phase D - Multiplayer
Phase E - Publishing Platform
Phase F - Public UGC Readiness
Phase G - Production Launch

Every phase ends with working source code, automated tests, staging deployment where applicable, documentation, benchmark results, known-risk report and formal acceptance demonstration.

SPEC 01 - Product Requirements

Status: Normative

Purpose: Define user roles, major product flows, tabletop primitives, creator workflows and MVP scope.

01.1 Roles and identity

Roles: Player, Spectator, Host, Creator and Moderator. Host is administrative only. Creator is an authenticated user owning a Project. Project collaboration is not required in MVP.

PRD-ID-001 - Guest participation - MUST

Guests can join invite-only tables.

PRD-ID-003 - Account-required actions - MUST

Authentication is required for persistent Projects, publishing, Library, persistent Saves and hosting public discoverable tables.

PRD-ID-004 - Internal identity - MUST

Use internal Account/Player IDs rather than spreading external auth-provider identifiers through domain objects.

01.2 Primary navigation and routes

Primary actions: Join, Host, Browse, Create. Home remains simple and action-oriented rather than becoming a social content feed.

```
/
/join/:code
/games
/games/:slug
/tables
/library
/saves
/create
/create/:projectId
/settings
```

01.3 Join and host

PRD-JOIN-001 - Join code - MUST

Join by human-readable room code.

PRD-JOIN-002 - Invite URL - MUST

Every joinable room has a shareable invite URL.

PRD-JOIN-003 - Input normalization - SHOULD

Reasonable casing/whitespace variations are normalized.

PRD-JOIN-005 - No unnecessary lobby - MUST

Successful join normally enters the actual Table.

PRD-JOIN-006 - Loading progress - MUST

Joining visibly shows connection/release/asset/start progress.

PRD-JOIN-009 - Full room - MUST

Full room rejects players and may offer spectating.

PRD-HOST-002 - Visibility - MUST

Rooms support Invite Only and Public.

PRD-HOST-004 - Game-defined settings - MUST

Games declare host-configurable boolean/number/choice settings.

PRD-HOST-006 - Host departure - MUST

Original Host departure does not end the Table.

01.4 Table interaction

PRD-TABLE-001 - Table-first UI - MUST

The 3D Table remains the dominant surface.

PRD-INT-001 - Direct manipulation - MUST

Grabbable world objects are draggable directly.

PRD-INT-002 - Immediate local response - MUST

Visual local grab does not wait for canonical acknowledgement.

PRD-INT-003 - Context menu - MUST

Objects expose capability-driven context actions.

PRD-INT-005 - Inspect - MUST

Eligible cards/models have local Inspect without canonical mutation.

PRD-INT-006 - Remote smoothness - MUST

Transient remote movement is interpolated.

PRD-GRAB-001 - Exclusive hold - MUST

A grabbable object has at most one canonical holder.

PRD-GRAB-002 - Conflict ordering - MUST

Simultaneous grabs resolve through canonical action ordering.

PRD-GRAB-004 - Disconnect cleanup - MUST

Logical departure cannot strand objects held forever.

01.5 Native tabletop semantics

Primitive	MVP behavior
Card	Front/back presentation, definition data, canonical face state, stacking
Deck	Ordered container, deterministic shuffle, draw, deal, insert top/bottom
Hand	Private official-client presentation, canonical contents, local visual sort by default
Container	Exclusive membership, optional capacity, deterministic ordering
Die	Deterministic RNG result, local animation
Counter	Canonical numeric value, set/add, optional bounds
Zone	Box/sphere, tag filtering, semantic enter/leave
Snap Point	Tag filtering, capacity, deterministic nearest candidate + stable-ID tie break
Button	Canonical press semantics
Text	Canonical text value when game logic changes it
Seat	Stable player/seat identity and optional default hand

01.6 Presence, reconnect, saves

PRD-PRES-001 - Remote cursor - SHOULD

Transient remote cursors should be supported.

PRD-PRES-003 - Laser pointer - SHOULD

Transient laser pointer should be included if low-cost.

PRD-REC-001 - Automatic reconnect - MUST

Loss of room WebSocket triggers automatic reconnect.

PRD-REC-002 - Same player identity - MUST

Reconnect restores same PlayerId where session remains valid.

PRD-REC-004 - Catch-up - MUST

Reconnect recovers via retained actions or checkpoint + replay.

PRD-SAVE-001 - Save canonical state - MUST

Authenticated Host can create persistent Saved Table.

PRD-SAVE-002 - Release pinning - MUST

Save retains exact Release needed to resume.

PRD-SAVE-004 - Transient exclusion - MUST

Save excludes camera/cursor/hover/WebRTC state.

01.7 Browse, library, create, publish

PRD-BROWSE-001 - Published games - MUST

Provide public discovery for published Games.

PRD-LIVE-001 - Public table discovery - MUST

Public live Tables are discoverable separately from Game records.

PRD-GAME-003 - Rules - MUST

Published Games support Overview, Setup and How to Play.

PRD-LIB-001 - Library - MUST

Authenticated users can bookmark published Games.

PRD-CREATE-002 - New project - MUST

Create from Blank Table, Card Game, Board Game or Dice Game.

PRD-PUB-001 - Publish validation - MUST

Publishing validates schema, Lua, refs, assets and runtime initialization.

PRD-PUB-004 - Immutable output - MUST

Successful publishing creates an immutable Release.

PRD-PUB-006 - Existing rooms unchanged - MUST

New Release publication does not change existing rooms.

01.8 Creator scripting and prompts

PRD-LUA-001 - Game scripts - MUST

Editor supports game-level Lua scripts.

PRD-LUA-002 - Entity scripts - MUST

Editor supports scripts attached to Entities/Prefabs.

PRD-LUA-003 - Script properties - MUST

Scripts have editor-configured props.

PRD-PROMPT-001 - Structured prompts - MUST

Lua can request choice, confirm and number prompts.

PRD-PROMPT-002 - Canonical response - MUST

Prompt response becomes canonical gameplay action.

PRD-SET-001 - Creator-defined settings - MUST

Creator defines host-configurable settings without custom room UI.

01.9 Playtest and online test

PRD-PLAYTEST-001 - Local playtest - MUST

Creator runs current project through actual Game Kernel.

PRD-PLAYTEST-002 - Snapshot build - MUST

Playtest runs from a frozen project snapshot/build.

PRD-PLAYTEST-003 - Isolation - MUST

Stopping playtest restores unchanged authored Project.

PRD-TEST-001 - Temporary Release - MUST

Online Test builds an immutable temporary Release.

PRD-TEST-004 - Cleanup - MUST

Temporary Releases have cleanup policy and do not appear in public Browse.

01.10 Sandbox, mobile, accessibility

PRD-SAND-001 - Interaction mode - MUST

Games declare scripted or sandbox interaction mode.

PRD-SAND-002 - Sandbox tools - SHOULD

Sandbox should support permission-controlled duplicate/delete/lock/spawn approved Prefabs.

PRD-MOB-001 - Responsive player UI - MUST

Player runtime is responsive to phone/tablet layouts.

PRD-MOB-003 - Editor exclusion - MUST

Mobile creator editor is out of MVP.

PRD-A11Y-001 - Conventional UI accessibility - MUST

Standard web UI targets WCAG 2.2 AA where practical.

PRD-A11Y-002 - Keyboard access - MUST

Forms, dialogs, menus, browse, publishing and editor panels support keyboard navigation.

01.11 Explicitly noncanonical state

```
camera
hover
pointer position
visual selection
interpolation
particles
animation progress
local hand sort order
inspect zoom
editor layout
audio playback position
remote cursor
laser pointer
WebRTC connection state
chat history
```

SPEC 02 - UX & Interaction Specification

Status: Normative

Purpose: Define player and creator behavior precisely enough that implementation and QA do not invent workflows.

02.1 UX principles

UX-001 - Table first - MUST

The 3D tabletop is the visual priority during play.

UX-002 - Progressive complexity - MUST

Player-facing interaction is substantially simpler than creator-facing interaction.

UX-003 - Direct manipulation - SHOULD

Prefer direct manipulation where the object metaphor supports it.

UX-004 - Recoverable actions - SHOULD

Ordinary reversible editor actions rely on Undo rather than unnecessary confirmations.

UX-005 - Explicit destructive actions - MUST

End Table, Project deletion, kick and public-content removal require confirmation.

UX-006 - Technical failures hidden by default - MUST

Players see human-readable errors; low-level sequence/hash/socket/Lua details stay in diagnostics.

02.2 Home, join and loading

```
HOME
[ JOIN TABLE ] [ HOST GAME ]
[ BROWSE GAMES ]

[ CREATE A GAME ]
```

Join accepts room code or supported invite URL, normalizes formatting, resolves the room, requests guest display name if required, and proceeds directly to the Table. Loading visibly advances through connecting, loading release/assets and starting simulation. Missing, ended, full and failed rooms have distinct recovery actions.

02.3 Table layout

```
+-----+
| Game Name                               Players Invite ... |
|                                     |
|                                     3D TABLE                 |
|                                     |
+-----+
|                                     |
|                                     HAND (if present)          |
|                                     |
+-----+
```

Top bar is minimal. Player panel shows display name, seat and connection state. Host-only actions include kick/seat assignment. Invite action exposes code and copy-link. Platform menu includes Rules, Settings and Leave; Host may also End Table and Save.

02.4 Object interaction

- Click selects; clicking empty table clears selection.
- Hover may show subtle outline/tooltip but must respect visibility.
- Pointer drag predicts grab immediately, lifts the object, starts transient movement, and submits canonical grab.
- Release submits canonical Drop transform; canonical resolution may remain in world, snap, enter zones or stack.
- Right-click/long-press opens capability-driven context menu.
- Inspect is local-only: large card image or rotatable model preview.
- Remote transient movement is interpolated; canonical correction uses a brief visual transition where possible.

02.5 Cards, decks, hands and dice

Deck supports Draw to Hand, Draw to World by dragging from top, Shuffle, Deal and Inspect. The client never chooses which hidden card is drawn; the Kernel derives it. Hand is a polished built-in private surface supporting inspect and drag-to-table; local visual sorting does not normally change canonical order. Dice expose Roll; the Kernel chooses the result and local physics/animation only visualizes it.

02.6 Camera and touch

Desktop baseline: orbit, pan, zoom, focus object/table. Touch baseline: tap select, drag selected object, long-press context menu, two-finger camera, pinch zoom. Actual mapping is validated in Phase A usability testing.

02.7 Reconnect and rollback UX

Transport loss keeps the Table visible behind a reconnect overlay. Interaction requiring canonical state pauses. Recovery catches up or resynchronizes without asking the user to understand snapshots. Prediction losses reconcile smoothly; messages may say another player acted first, never "rollback frame".

02.8 Creator dashboard and Editor

Create landing shows New Game and Projects. Editor default layout: Hierarchy/Project left, Scene/Code tabs center, Inspector right, Console/Problems/Runtime/Network bottom. The center workspace allows Main Scene, Prefabs and Lua scripts as tabs. Autosave status is visible.

02.9 Publishing UX

```
PUBLISH
1. Checking game
  - Scripts
  - Assets
  - References
  - Runtime
  - Performance warnings
2. Ready to Publish
  - Visibility
  - Release notes
3. Success
  - View Game
  - Copy Link
```

02.10 Usability acceptance

Human usability testing must cover join friend, camera, grab/move, draw/play card, hand, die, inspect, rules, invite and reconnect. Creator testing covers create project, add objects, configure deck, create stable ref, write/fix Lua, playtest, online test and publish.

SPEC 03 - Deterministic Game Kernel

Status: Normative architecture

Purpose: Define canonical gameplay semantics independent of UI, rendering and Cloudflare implementation.

03.1 Definition

For valid initial snapshot S , ordered action stream A and kernel version K :

$$\text{run}(K, S, A) = \text{identical canonical state}$$

KRN-DEP-001 - Dependency exclusions - MUST

Kernel has no React, Babylon.js, DOM, WebRTC, Cloudflare APIs, browser wall clock, localStorage, IndexedDB, Math.random or presentation physics dependencies.

KRN-DEP-002 - Headless execution - MUST

Kernel runs in a headless Bun/Node-compatible test environment without DOM shims.

03.2 Canonical state

```
CanonicalGameState {
  schemaVersion
  sequence
  releaseId
  kernelVersion
  settings
  rng
  players
  seats
  entities
  scriptState
  prompts
}
```

Canonical values are finite JSON-like primitives, arrays and string-keyed objects. Functions, Promises, DOM/Babylon objects, VM pointers, cyclic graphs and unsupported nonfinite numbers cannot enter canonical state.

03.3 Ordered actions and rejection

Every service-accepted OrderedAction consumes one monotonically increasing Sequence. Game-semantic rejection still advances processed Sequence but does not mutate gameplay fields. All top-level action processing is transactional: if a script or generated subcommand fails, all uncommitted gameplay mutation is discarded.

03.4 Mutation boundary and registry

Canonical state changes only via a registered canonical action or deterministic subcommand generated during the current transaction. Each action has type/version, allowed sources, payload schema, validation/application logic and event metadata. Player, script and system sources are validated independently of UI visibility.

03.5 Identity, transforms and location

Entity IDs are immutable and never reused after destruction. Spawned IDs derive deterministically from canonical action identity plus spawn index. Canonical transforms reject NaN/Infinity/invalid quaternion/out-of-bounds values and are normalized/quantized before hashing. High-frequency drag transforms are transient; only Grab and final Drop are canonical.

03.6 Tabletop invariants

- Container membership has one canonical source of truth.
- Deck is Container semantics + Deck capability; last item is top in v1.
- Shuffle uses deterministic RNG and never accepts a client-provided resulting order.
- Draw/deal derive card identity from canonical Deck contents; insufficient requested cards reject transaction.
- Hand is Container + owner/visibility semantics; visual sort is local unless canonicalOrder enabled.
- Die Roll consumes deterministic RNG; animation never chooses canonical result.
- Counter set/add clamps to optional bounds.
- Zone membership is recomputed on semantic placement transitions, not frame physics.
- Snap resolution filters valid candidates, picks nearest, then stable SnapPoint ID on exact tie.

03.7 RNG, serialization and hash

RNG algorithm is explicit, versioned and snapshot-visible. Gameplay RNG is separated from presentation randomness. Canonical serialization defines UTF-8 representation, object key ordering, number normalization, array ordering, null semantics and unsupported-value rejection. SHA-256 is the baseline StateHash. Hash is over canonical serialization, not compressed bytes.

03.8 Snapshot, replay, rollback and prediction

```
GameSnapshot { formatVersion, kernelVersion, releaseId, sequence, state, stateHash }
```

Snapshot load must reproduce the stored hash. Rollback implementation may use local snapshots, patches or persistent structures, but must converge to the same final hash as uninterrupted canonical execution. Prediction is initially recommended for grab/drop/flip/button/simple counter/prompt responses, and conservative for random/large scripted actions.

03.9 Lua and timers

Canonical Lua executes inside action transactions and can only queue deterministic Kernel mutations. Persistent Lua state is Kernel-owned. Script errors abort the top-level transaction. Execution budgets limit instructions/operations, memory, generated commands, recursion and persistent state size. Timers are scheduled by room service and become canonical system.timer_fire actions.

03.10 Kernel acceptance invariants

```
same snapshot + actions = same hash
snapshot/load preserves hash
rollback/replay = uninterrupted execution
rejected action has no partial gameplay mutation
RNG replay identical
destroyed IDs never reused
exclusive Entity in at most one container
held Entity has at most one holder
invalid transform cannot enter state
script failure atomic
query and generated-command order deterministic
```

SPEC 04 - Lua API v1

Status: Normative creator API

Purpose: Define the stable scripting surface and sandbox expectations for deterministic tabletop rules.

04.1 Philosophy and environment

Lua expresses tabletop rules, not browser/renderer implementation. The v1 environment exposes state, refs, settings, game, scene, players, random, timer, ui and data. Entity scripts additionally receive self and props. Unsafe browser/network/storage/filesystem/OS and arbitrary JavaScript access are unavailable. `require()` resolves only packaged project/platform modules.

```
refs.main_deck:shuffle()
refs.score:add(1)
ctx.player.hand:add(card)
self:flip()
```

04.2 Persistent state

If a value affects future gameplay, it belongs in state. Persistent values are canonical-compatible nil/boolean/finite number/string/tables. Functions, userdata, proxies, cycles and nonfinite numbers cannot persist. Fresh VM + Release code + canonical state must reconstruct gameplay behavior.

04.3 refs, settings, props and self

- refs resolves stable editor-defined Entity references by identity; rename does not break it.
- settings is read-only canonical room-start configuration.
- props contains editor-authored per-binding configuration.
- self is the Entity proxy for an entity-bound script. Destroyed proxies become invalid safely.

04.4 Core namespaces

```
scene:get(id)
scene:find(name)
scene:query({tags={"enemy", "unit"}})
scene:spawn(prefabId, options)
scene:destroy(entity)

players:list()
players:get(id)
players:count()
players:by_seat(seatId)

random:int(min, max)
random:float()
random:choice(list)
random:shuffle(list)
```

All collection/query ordering that can affect canonical scripts is explicitly deterministic. Recommended `scene:query` ordering is ascending stable EntityId; player ordering uses stable seat order then PlayerId for unseated participants.

04.5 Semantic proxies

Proxy	Required v1 methods/fields
Card	is_face_up, flip, set_face_up, definition_id, definition
Deck	count, shuffle, draw_to, draw_to_world, deal, insert_top, insert_bottom
Hand	count, list, add, remove, contains
Container	count, list, contains, add, remove, move_to
Die	roll, value, faces, script-only set_value
Counter	value, set, add, subtract, reset
Zone	contains, entities
SnapPoint	is_occupied, entities, attach, detach
Button	is_enabled, set_enabled, set_label
Text	get, set

Player	id, name, connected, role, seat, hand, message
--------	--

04.6 Timers and prompts

```

timer:after(5, "finish_turn")
timer:cancel(timer_id)

ui:prompt(player, { id="action", title="Choose", choices={...} })
ui:confirm(player, { id="end_turn", title="End your turn?" })
ui:number_prompt(player, { id="bid", min=0, max=10, step=1, default=3 })

```

Timer callbacks are named, not anonymous closures, so they survive reconstruction. Prompt state is canonical and response becomes `prompt.respond`. v1 deliberately does not expose a general DOM/custom-UI framework.

04.7 Callbacks and guards

```

Game: on_start, on_player_join, on_player_disconnect, on_player_removed, on_prompt
Entity: on_click, on_grab, on_drop, on_flip, on_spawn, on_destroy, on_roll, on_press, on_enter, on_leave,
on_container_add, on_container_remove
Guards: can_click, can_grab, can_drop, can_flip, can_press

```

Guards are read-only. They may allow/deny and optionally return a user-facing reason, but cannot queue canonical mutations. Event subscribers execute in deterministic stable `ScriptBindingId` order. Normal event callbacks may queue subcommands FIFO inside the parent transaction.

04.8 Failure and budgets

Canonical script errors record `script/binding/function/line/message/Sequence` and abort the top-level transaction. Infinite loops, recursion, command explosion and memory growth are bounded. Wall-clock timeout alone is not sufficient as the sole deterministic guardrail. Phase A must prove a viable instruction/operation budget strategy.

04.9 Example

```

function on_start(ctx)
  state.turn = 1
  refs.main_deck:shuffle()
  for _, player in ipairs(players:list()) do
    if player.hand then refs.main_deck:deal(player, 5) end
  end
end

function can_grab(ctx)
  return ctx.player.seat.id == props.seat_id, "Only this seat may move this piece."
end

```

SPEC 05 - Multiplayer Protocol & Room Service

Status: Normative realtime protocol

Purpose: Define room lifecycle, sequencing, reconnect, prediction, checkpoints, timers, WebRTC and failure handling.

05.1 Core principle and lifecycle

Client requests -> Room Durable Object -> Canonical order -> Every client runs identical Game Kernel

CREATING -> ACTIVE -> EMPTY -> ENDED

Identifiers are distinct: RoomId, JoinCode, PlayerId, ConnectionId, RoomSessionToken, RequestId, ActionId, Sequence, CheckpointId and TimerId. A reconnect creates a new Connection for the same Player.

05.2 Create, join and session

Room creation validates Release readiness/permission/settings, allocates RoomId and JoinCode, chooses canonical seed, initializes room state and joins Host. Join validates room/capacity/spectator policy and returns RoomId, PlayerId, session token, ReleaseId and WebSocket endpoint. Session tokens are unguessable, room/player scoped, revocable and never public.

05.3 Handshake and recovery

```
hello { protocolVersion, sessionToken, lastSequence }

response:
bootstrap | resume | resync_required | protocol_error | room_ended
```

Recent reconnect receives missing actions. Stale or divergent client loads a confirmed checkpoint, verifies hash, replays later actions, then resumes. Sequence gaps are detected; clients never silently skip missing canonical actions.

05.4 Canonical action request and sequencing

```
action_request { requestId, predictedAtSequence, action:{type,payload} }

ordered_action { sequence, actionId, requestId?, actor, action }
```

Room service validates protocol/session/rate/size but does not run game-specific Lua or semantic validation. Actor identity comes from authenticated room session, not client payload. Each unique service-valid request receives exactly one Sequence. Duplicate RequestId returns/rebroadcasts the original mapping and never creates a second action.

05.5 Prediction and reconciliation

Client maintains confirmed state plus pending local predictions. Matching requestId confirms placement in the canonical stream. If intervening canonical actions invalidate a prediction, client rolls back to a confirmed point, applies canonical actions, then replays still-valid predictions. Final canonical hash must match clean canonical execution.

05.6 Checkpoints and hash consensus

Room retains a bounded recent action window. Client-generated checkpoints include sequence/hash/snapshot/release/kernel version. With multiple healthy clients, room seeks at least one additional matching hash before confirming a checkpoint where practical. v1 is friendly-mode, not Byzantine consensus. Hash mismatch triggers automatic resync and telemetry.

05.7 Disconnect, seats and host

Transport loss first marks a Player reconnecting; no game-level disconnect action is emitted until grace expiry or explicit leave. Canonical player departure releases held objects. Host is room-service metadata, normally migrating to the oldest connected non-spectator. Host identity is not exposed as v1 gameplay state. Kick revokes session and emits canonical player.removed.

05.8 Timers and empty rooms

Room service schedules canonical timers from Lua registrations. Due timers become system.timer_fire exactly once. One-shot timers survive service restart without duplicate firing. Empty rooms pause ordinary game timers by default; room expiration is a configurable operational policy, baseline around 30 minutes.

05.9 WebRTC transient path

WebRTC carries only transient drag previews, remote cursors and laser pointers. Full mesh is the 1-8 player baseline subject to Phase A. Transient channel is unordered/unreliable/latest-state-wins. If unavailable, transient data may use a rate-limited WebSocket fallback. Disabling WebRTC entirely must not prevent correct game completion.

05.10 Public room discovery, chat and saves

Public rooms are indexed in queryable metadata rather than enumerating DOs. Index is discovery cache only; actual join validates against Room. Basic text chat is room-service state, not rollback state. Save creates a persistent canonical snapshot pinned to exact Release; MVP Save creation is Host-only and requires an authenticated Host. Resume creates a new live Room from the Save.

05.11 Required network tests

```
8 clients
latency + jitter
duplicate requests
missing packets
random reconnects
DO restart
WebRTC loss
Host departure
mid-grab disconnect
timer restart
save/resume
=> all healthy clients converge to identical final hash
```

SPEC 06 - Editor & Creator System

Status: Normative editor specification

Purpose: Define project state, persistence, undo/redo, prefabs, assets, scripts, playtest and publishing behavior.

06.1 State boundaries

```
Project State = persistent authored content
Editor State = selection/tabs/layout/camera/gizmo preview
Runtime State = active Game Kernel playtest
Persistence State = revision/saving/conflict/offline
```

All authored mutations pass through editor-core. React requests commands; Babylon reports interactions and projects state visually. Neither directly owns Project persistence.

06.2 Commands, undo and autosave

Every authored structural/property change is an EditorCommand or transactional group. One semantic Add Card or transform drag produces one Undo step. Runtime Playtest actions never enter editor Undo history. Autosave uses monotonically increasing Project revisions and expectedRevision conflict protection.

06.3 Single-writer and recovery

MVP allows one active write lease per Project. A second editor can open read-only or take over, invalidating the prior writer. Lease loss is visible and unsaved local changes are recoverable rather than silently discarded. Browser local recovery buffer protects against crash/network loss; full offline-first editing is out of scope.

06.4 Hierarchy and Inspector

Hierarchy operates on stable Entity IDs and supports select, rename, reparent, duplicate, delete, search and context actions. Display name is never identity. Reparent preserves world transform by default. Ordinary component inspectors are schema-driven; rich components such as Deck, Seat, Script and Game Settings may use custom inspectors.

06.5 Prefabs

Prefab is a reusable Entity hierarchy with stable local IDs. Instances retain source linkage plus explicit overrides. Updating a Prefab updates inherited values but does not erase overrides. Orphaned overrides are surfaced. Unpack Prefab converts to ordinary independent Entities. v1 prohibits nested live Prefab instances to avoid override/cycle complexity.

06.6 Assets

Asset states: Uploading, Processing, Ready, Failed, Archived. Supported MVP imports: GLB and PNG/JPEG/WebP. Uploads are untrusted and server-side processed/validated. Release references immutable processed runtime content, not raw uploads. Models cannot cause arbitrary runtime fetches to creator-controlled origins. Asset deletion never deletes bytes still referenced by Releases/Saves.

06.7 Scripts and diagnostics

Lua scripts have stable ScriptId independent of filename. Entity script bindings have stable BindingId and props. CodeMirror provides Lua highlighting, line numbers, search, folding, autocomplete, API hover and diagnostics. Problems are structured Error/Warning/Info entries; Runtime diagnostics show Sequence, StateHash, recent actions, script errors, prompts, timers and RNG summary.

06.8 Build, playtest and online test

```
Validate -> Resolve Scenes -> Resolve Prefabs -> Resolve Refs -> Validate Components/Settings/Seats ->
Validate Lua -> Validate Assets -> Normalize Runtime -> Generate Initial Template -> Manifest -> Hash
```

Local Playtest freezes a Project revision/snapshot and uses the same runtime concepts as Online Test and Published play. Editing is read-only while Playtest runs in v1. Stopping disposes runtime and restores unchanged authored state. Online Test builds an immutable temporary unlisted Release and normal room; editing requires stopping/rebuilding rather than hot-mutating the running deterministic session.

06.9 Publish transaction

Publish captures Project Revision N, validates/builds immutable artifacts, writes Release manifest/hashes, marks Release Ready, then updates Game.latestReleaseId. Creator may continue editing N+1 while N publishes. Later Draft changes cannot alter the Release. Release bytes/scripts/settings/compatibility metadata remain immutable after Ready; only operational status may change.

SPEC 07 - Platform, Data, Security & UGC

Status: Normative platform/security specification

Purpose: Define storage authority, ownership, release lifecycle, authorization, moderation, retention and threat boundaries.

07.1 Authority map

```
Account/Game/Project metadata -> Platform Worker + D1
Project/Release/Asset blobs -> R2 + metadata
Active Room order -> Room Durable Object
Gameplay State -> deterministic client Kernel reconstruction
Public Room discovery -> queryable index
Transient movement -> WebRTC / socket fallback
```

07.2 Core relational model

```
User
Project -> ProjectRevision / ProjectWriteLease
Game -> Release
Asset
LibraryItem
SavedTable
PublicRoom
Tag / GameTag
Report
ModerationAction
AuditEvent
```

D1 is relational/queryable metadata, not per-frame simulation or large binary storage. R2 is baseline for larger immutable/project/release/snapshot objects. DO in-memory state must be reconstructable across hibernation/restart from persistent state and durable references.

07.3 Game, Release and Asset lifecycle

Game is stable public identity and metadata. Release is immutable playable build with ReleaseId, ReleaseNumber, manifest hash, Kernel/Lua/format versions and operational/moderation status. Publish builds artifacts first and updates Game.latestReleaseId only after Release becomes Ready. Asset records track source and processed blobs; published releases reference processed immutable content.

07.4 Authorization

Backend validates every ownership/admin action. Guests may join invite rooms and read public Games. Project read/edit/publish require owner permission. Host can perform room administration. Moderators perform privileged disable/suspend/restore. Client-provided ownerId, PlayerId or Host flags are never authorization.

07.5 Save ownership

MVP locks Save creation to the administrative Host. Save owner is the authenticated Host account. A guest Host must sign in before persistent Save creation. Any-participant Save forks are deferred.

07.6 Security boundaries

- UGC text is untrusted and rendered as plain text or strictly sanitized restricted Markdown.
- Production targets restrictive CSP; creator content cannot require arbitrary script/iframe origins.
- Lua is hostile input: bound execution/memory/commands/recursion; no browser/network escape.
- Assets are hostile input: size, format, structure, remote dependencies and resource use are validated.
- Room abuse: join-code guessing, action flood, reconnect flood, chat/RTC spam, oversized payload and snapshot abuse are rate-limited.
- Room session tokens are high-entropy, scoped, revocable, never logged/public.
- Release integrity chain: ReleaseRecord -> manifestHash -> manifest -> file hashes.
- Checkpoint integrity uses canonical StateHash, not object-key trust alone.

07.7 Join-code security

Join-code format remains a security design decision. It must be human-readable, rate-limited and provide adequate effective entropy. Illustrative BIRD-742 formatting must not be implemented without calculating real code-space entropy; recommendation is roughly 30+ effective bits plus rate limiting.

07.8 Moderation

Public UGC launch requires Report Game/User, moderator queue, restrict/restore Game, disable/restore Release, suspend/restore User and append-only moderation audit. Hard Release disable blocks new rooms/resumes and may terminate active rooms only for severe incidents. Creator unpublish changes discoverability but does not destroy immutable data referenced by Saves.

07.9 Retention baseline

Data class	Baseline
Temporary Online Test Release	~7 days after last relevant dependency/activity
Ended unsaved room data	24-72 hours operational/debug grace
Project soft delete	~30 days
Room chat	Ephemeral/short retention
Published Release referenced by Save	Retain
Moderation audit	Long-lived subject to policy/legal review

07.10 Environment and ownership

At least local, staging and production environments have separate D1/R2/DO/identity/secrets. Production resources live in client-controlled accounts. Repository contains deployment config, DO/D1 migrations, environment documentation and runbooks. No recurring production step may depend on undocumented vendor knowledge.

SPEC 08 - Acceptance, Delivery & Traceability

Status: Normative acceptance framework

Purpose: Tie every MUST requirement to implementation evidence, tests, milestones and payment gates.

08.1 Traceability

Requirement ID	Title	Priority	Spec	Section	Phase	Implementation	Test ID	Test Class	Status

Proof classes: UNIT, INTEGRATION, SIMULATION, E2E, HUMAN, SECURITY and PERFORMANCE. A screenshot cannot substitute for deterministic/system proof.

08.2 Executable-style acceptance

```

GIVEN known starting state
WHEN specific action/environment event
THEN observable required result
  
```

08.3 Minimum acceptance suites

Area	Required examples
Kernel	cross-runtime determinism, script-error atomicity, RNG replay, container invariant, destroyed ID, rollback convergence, zone/snap determinism, prompt validation
Lua	no browser globals/JS/network, infinite-loop interruption, command explosion, fresh-VM state recovery, read-only guards, refs survive rename, stable query order
Editor	semantic undo, transform single undo, Playtest isolation, revision conflict, writer lease, Prefab override, invalid GLB, stable ref, captured-revision Publish
Multiplayer	sequence continuity, DO restart, gap detection, recent/stale reconnect, divergence resync, WebRTC-disabled play, timer restart, host leave, kick token revocation
Publishing	immutable release, old room stays old release, missing ref/asset blocks, integrity failure, Save pins Release, old fixture compatibility
Security	authz, XSS, Lua sandbox, asset abuse, room abuse, session revocation, dependency scanning, manual review

08.4 Performance fixtures

```

PERF-SMALL 100 Entities
PERF-MEDIUM 1,000
PERF-LARGE 5,000
PERF-TABLE 250 visible objects
PERF-MULTI 8 connected players
PERF-ROLLBACK 100/500 action windows
  
```

Phase A converts conceptual targets into agreed reference hardware/device classes and exact pass/fail numbers. Later phases are measured against those approved budgets.

08.5 Phase gates

Phase	Gate question
A Validation	Are the risky architecture premises proven or explicitly redesigned?
B Core Runtime	Is canonical simulation proven independently of renderer/network?
C Creator + Local Play	Can a creator build and locally test a small game without raw JSON?
D Multiplayer	Can several real users play/reconnect/recover without divergence or Host dependency?
E Publishing	Can an unrelated creator publish and an unrelated user discover/host that exact immutable Release?
F UGC Readiness	Can public unknown creators be supported safely?
G Launch	Can production operate, recover and be handed over independently?

08.6 Deliverables and handover

Every phase delivers source in client repo, tests, fixtures, schemas/API docs, deployment config, staging deployment, architecture update, known issues and benchmark reports where relevant. Final handover includes all source, Cloudflare resources, identity, domains/DNS, CI/CD, monitoring, design files, migrations, admin tools, fixtures, protocol/Lua docs and runbooks.

SPEC 09 - Statement of Work & Vendor Procurement Requirements

Status: Normative procurement specification

Purpose: Define engagement structure, vendor responsibilities, ownership, bid format and acceptance conditions.

09.1 Engagement objective

The selected vendor designs, validates, implements, tests, deploys, documents and hands over the platform in accordance with SPEC 00-08 and appendices. Vendor should challenge technically unsound assumptions through evidence and ADRs, but must not silently substitute a materially different architecture for convenience.

09.2 Phase-by-phase deliverables

Phase A delivers working validation prototypes and a measured go/conditional-go/redesign/stop report. Phases B-G deliver the runtime, editor, multiplayer, publishing, UGC hardening and launch/handover as specified. Source code and tests are required at each phase; slide-only milestones are not acceptable.

09.3 Vendor competencies

- Technical architecture
- Complex React applications
- Browser 3D/Babylon or equivalent
- Game/state simulation
- Realtime networking
- Cloudflare Workers/DO or equivalent edge state
- Lua/WASM sandboxing or embedded scripting
- Automated testing
- Product/interaction design
- Security
- Production operations

09.4 Proposal questions

- Top five technical risks and why.
- Which LOCKED decisions would you challenge?
- Which BASELINE decisions would you validate first?
- How will you prove deterministic equivalence across browsers?
- How will you bound Lua and prevent partial mutation after failure?
- How will prediction/rollback work for conflicting grabs?
- How will DO restart/reconnect preserve order?
- How will old Releases remain compatible?
- How will UGC assets/scripts be isolated?
- How will Babylon scenes and WebRTC/TURN be benchmarked?
- What would make you redesign after Phase A?
- What assumptions materially affect estimate?

09.5 Commercial and ownership expectations

Estimate Phase A-G separately with team, effort range, calendar range, assumptions, exclusions, external costs and risk. Client controls source repos, Cloudflare resources, identity provider, domains, CI/CD and monitoring. Third-party dependencies/licenses are disclosed. No undisclosed proprietary vendor runtime dependency is acceptable.

09.6 Acceptance and defect severity

Phase acceptance is based on executable deliverables and SPEC 08 evidence. Critical defects include security compromise, data loss, canonical divergence corruption, unrecoverable deploy/recovery failure or unauthorized private data access. No known Critical defect may remain at final acceptance. High defects require explicit client acceptance to defer.

SPEC 10 - Finished UX & Creator DX Contract

Status: Normative

Purpose: Define the finished creator/player experience so implementation follows an already-designed product rather than discovering UX during paid engineering iterations.

10.1 Product experience principles

Players should understand basic tabletop interaction within seconds. Creators should build useful games before writing Lua. Programmers should feel like they are scripting a tabletop rather than wrapping a rendering engine.

The creator abstraction hierarchy is: tabletop objects -> behaviors -> game rules/settings -> Lua -> raw components/advanced diagnostics.

Semantic/simple inspectors are the default. Raw component composition is an Advanced authoring surface.

10.2 Intent-driven authoring

The Add menu is organized around Cards, Pieces, Random, Game Logic and Scene objects. Objects such as Pawn are editor abstractions that expand into reusable component bundles rather than new Kernel primitives.

Inspectors present semantic settings first: a Deck shows cards, back, start behavior, player permissions and primary action before exposing Container/Deck/Collider components.

Executable starter templates are required: Card Game, Board Game, Dice Game and Blank Sandbox must be playable immediately after creation.

10.3 Declarative behavior layer

v1 includes a deliberately small no-code behavior system using WHEN / optional IF / THEN. It is not a general visual programming environment.

Recommended trigger set: Pressed, Entered Zone, Left Zone, Flipped, Rolled, Dropped, Game Started and Turn Started.

Recommended condition set: tag check, seat/player check, counter comparison, die-result comparison and game-setting comparison.

Recommended action set: counter set/add, message, move/destroy/flip object, draw/deal cards, enable/disable button, set text, advance turn and end game.

10.4 Single runtime semantics - Behavior AST to Lua

LOCKED: the Behavior Builder stores a restricted authored Behavior AST. The deterministic build compiler generates Lua from that AST; generated Lua runs through the normal Lua runtime and Game Kernel API.

There is no second visual-behavior runtime. Generated errors are source-mapped back to the Behavior UI.

Convert to Lua is one-way and is one undoable editor transaction. Arbitrary Lua is never parsed back into visual behaviors.

10.5 Lua standard library

A versioned deterministic Lua standard library sits above the Kernel API. Initial recommended modules are turns, rounds, scores and teams.

Releases pin luaStdlibVersion separately from kernelVersion and luaApiVersion. Convenience libraries do not become Kernel primitives merely to shorten creator code.

Recommended turns API: start, current, next, index, is_current and stop. Recommended scores API: set, add, get and leader.

10.6 CodeMirror DX

Autocomplete is project-aware: refs.* lists stable project references and proxy completion only exposes methods valid for the referenced semantic object.

Hover documentation includes signatures, determinism notes and examples. The API browser is available inside the editor/command palette.

Use in Script creates a stable reference if necessary and inserts refs.<name>. Contextual script creation can generate event callbacks such as on_press(ctx).

10.7 Creator diagnostic contract

Creator-facing errors answer: what happened, where, why and how to fix it. Raw Lua/engine traces are secondary advanced detail.

Diagnostics are domain-aware (for example, a Deck cannot roll) and Problems may provide undoable quick fixes such as Add Container, Choose Object or Optimize Asset.

Generated Behavior Lua errors must surface as Behavior errors rather than generated-file line noise.

10.8 Creator loops

Local loop: Edit -> Play -> Observe -> Stop -> Fix -> Play. Routine Play and Stop are one-click actions.

Online loop: Test Online -> immutable temporary build -> private table -> invite friend -> stop -> edit -> create new temporary build. Edits made while a test runs apply only to the next test.

Publish exposes a confidence checklist for game startup, scripts, references, assets, performance recommendations, multiplayer compatibility and Rules.

10.9 Live player interaction

First-interaction hints teach movement contextually and disappear after learning. Context menus show applicable semantic actions and shortcuts only.

Objects may have a creator-configured Primary Action (for example Die=Roll, Card=Flip, Deck=Draw, Button=Press), usable by double-click/tap where appropriate.

Holding is communicated in product language: if Bob holds an object, another player sees "Bob is holding this" rather than a protocol rejection.

10.10 Live undo, sandbox recovery and game lifecycle

LOCKED: no general multiplayer canonical Undo in v1. Rewinding history after scripts/RNG/timers is intentionally avoided.

Sandbox deletion may use recoverable Table Trash rather than history rewind. The finalized Trash semantics are defined in SPEC 11.

Canonical game lifecycle is INITIALIZING -> ACTIVE -> FINISHED. game:end(...) produces shared canonical results and ordinary gameplay becomes read-only after FINISHED while camera/inspect/chat remain available.

Generic seamless Restart Game is deferred because it introduces room/game epoch semantics.

10.11 Creator-facing vocabulary

Creator release vocabulary is Draft / Test / Published. Game visibility is Private / Unlisted / Public.

Community remixing, asset packs and marketplace functionality are OUT of MVP.

Account-synced, device-local and room/session-local settings must remain distinct; camera/hand presentation state never leaks into canonical game state.

10.12 Finished DX acceptance examples

DX-001: creator can modify a Card Game template into a functional deal flow without opening Lua.

DX-002: visual Zone behavior converts into readable equivalent Lua and still uses the same deterministic runtime.

DX-003: Use in Script creates stable ref that survives object rename.

DX-004: invalid semantic method produces capability-aware diagnostic rather than raw nil-method error.

DX-005: Problems quick fix executes as an undoable Editor command.

DX-006: Playtest gameplay mutations leave the Draft exactly unchanged after Stop.

10.13 Representative creator code

```
function on_enter(ctx)
    if ctx.object:has_tag("treasure") then
        refs.score:add(1)
        ctx.object:destroy()
    end
end

function on_start(ctx)
    turns:start(players:list())
    refs.main_deck:shuffle()
end
```

SPEC 11 - Screen-by-Screen UX & DX Contract

Status: Normative

Purpose: Define the main application screens, transitions, product-language recovery states and the final architecture of the Behavior compiler and sandbox recovery model.

11.1 Information architecture

Public/player surfaces: Browse, Game Detail, Join Table and Live Table. Account surfaces: Library, Saved Tables and Settings. Creator surfaces: Projects, Editor, Test Online and Publish. Admin surfaces: Reports, Moderation and operational diagnostics.

Players never need to understand Project revisions, Kernel versions or build artifacts. Creators see only the technical concepts needed to make and debug games.

11.2 Dashboard and Create Game

Dashboard prioritizes Your Projects, New Game, Recent Tables and Browse. Empty state immediately offers Create Game.

Create Game asks for a starter template and name, then opens the Editor immediately. Templates are executable working games rather than decorative starter scenes.

11.3 Editor shell

Default desktop editor provides persistent top-level Play, Test Online and Publish actions, a Hierarchy, central Viewport/Code document tabs, semantic Inspector and Problems/Console status area.

Hierarchy displays authored entities, never Babylon mesh nodes. Double-click frames an entity; drag reparents; search remains available.

A command palette provides expert access to Add, Play, Test Online, Publish, Open Script, Go to Object, Browse Lua API, Quick Fix and Reset Layout.

11.4 Behavior Builder compiler contract

Authoring stores BehaviorDefinition AST nodes with stable BehaviorId, Trigger, Conditions and Actions. Build deterministically compiles these definitions into generated Lua.

Generated Lua uses normal ScriptRuntime semantics and Kernel actions. Generated diagnostic source maps point back to the creator-facing Behavior node.

Convert to Lua creates a readable ScriptDocument/Binding, removes the Behavior definition and opens the generated source; the full conversion is one undoable Editor command.

11.5 Play and Online Test modes

Play mode visibly changes the editor state and disables ordinary authored mutation. Stop disposes runtime and restores the authored projection.

Test Online builds a private immutable temporary Release and presents Copy Invite/Open Table/Stop Test. Draft edits may continue but explicitly apply only to the next test build.

No deterministic hot reload is required.

11.6 Publish screen

Publish begins with a readiness checklist covering startup, scripts, references, assets, multiplayer compatibility, performance recommendations and Rules.

Publishing identifies the captured Project revision. Success shows Published Release number, View Game and Copy Link while the Editor continues on mutable Draft.

11.7 Join and Live Table

Guest invite flow is minimal: game identity, participant count, display name and Join Table. No account wall for invite-only guest play.

The live table prioritizes the 3D surface, with sparse game name/player/menu chrome, optional Chat drawer and bottom Hand. Platform navigation does not remain persistently visible during play.

Context menus are capability-driven; remote hold ownership and reconnect status use player language.

11.8 Hand, Inspect and responsive player UX

Hand is a platform projection of canonical Hand contents; local sorting remains presentation-only unless canonicalOrder=true.

Inspect is local/read-only and exposes safe game-facing information, not raw components.

Desktop, tablet and phone layouts all keep the Table dominant. The Editor remains desktop-only for MVP.

11.9 Sandbox Trash semantics

LOCKED: sandbox Trash is canonical soft removal, not entity.destroy. A trashed Entity remains alive with the same EntityId but is inactive, not rendered/interactable, absent from ordinary queries and detached from active Zones/Snaps/Containers as required.

TrashEntry records entityId, trashedBy, and enough original location/transform information for deterministic restore.

sandbox.restore reactivates the same EntityId. Permanent purge may later invoke true destroy.

This design preserves the Kernel invariant that destroyed Entity IDs are never reused.

11.10 Reconnect and divergence UX

Short network interruptions show Reconnecting while camera remains usable; canonical interactions pause. Reconnect within grace preserves Player identity and Seat.

Checkpoint recovery shows Synchronizing Table. Hash mismatch and protocol details remain creator/internal diagnostics.

Repeated synchronization failure produces a recoverable product error with Reload Table and Leave options.

11.11 Finished game, Game Detail, Browse, Library and Saves

game:end produces the same shared result overlay for all healthy clients; View Table dismisses the overlay while gameplay remains read-only.

Game Detail shows hero, player range, tags, Host, Library, description, Rules and creator identity. Host Game automatically renders creator-defined room settings.

Browse Games and Browse Live Tables are separate concepts. Library contains saved game bookmarks; Saved Tables pin the exact Release and resume via a new live Room.

11.12 Loading, errors and accessibility

Async transitions use specific status labels such as Loading Game, Joining Table, Synchronizing Table, Processing Model, Building Test and Publishing Revision rather than generic indefinite spinners.

Errors are classified as retryable, creator-fixable, permission, incompatible or terminal and provide the relevant next action.

Conventional UI requires logical tab order, visible focus, labels, keyboard menus/dialogs and reduced motion. Every transform manipulable by gizmo is also editable numerically in the Inspector.

11.13 Screen acceptance additions

UX-CREATE-001: creator reaches a working template game in one creation flow.

UX-EDIT-001: primary Deck configuration requires no Advanced component editing.

UX-BEH-001/002: Zone behavior works without Lua and converts to readable deterministic Lua.

UX-TEST-001: edits during Online Test do not mutate the running temporary Release.

UX-JOIN-001: guest joins invite table without Account.

UX-HAND-001: card move from Hand to world produces canonical location transition.

UX-NET-001: reconnect within grace requires no manual rejoin.

UX-END-001: all healthy players observe identical finished-game result.

UX-A11Y-001: gizmo-manipulated properties are keyboard editable.

11.14 Remaining UX validation questions

- Exact desktop camera input mapping - recommend right-drag orbit, modified/middle drag pan and wheel zoom, then finalize in usability testing.
- Phone classification - retain responsive target but allow Phase A to decide fully-supported versus best-effort precision interaction.

- Declarative behavior catalog size - recommend approximately 8-12 triggers, 5-8 conditions and 10-15 actions, avoiding drift into general visual programming.
- Terminology testing for Behavior, Test Online, Prefab and Reference.
- Table Trash remains SHOULD unless cost is trivial; its canonical semantics are settled if implemented.

Appendix A - Consolidated Decision Register

Status: Current truth

Purpose: Separate product/architecture contract from baselines and Phase A questions.

A.1 Status meanings

Status	Meaning
LOCKED	Product/architecture contract; change requires explicit approval/ADR.
BASELINE	Preferred implementation; vendor may recommend evidence-based change via ADR.
OPEN	Must be resolved during named validation/design phase.
OUT	Explicitly outside MVP.

A.2 Key locked decisions

- Browser-native player; guest invite joining; account-gated creator/persistent features.
- Deterministic Kernel independent of React/Babylon/network.
- Host is administrative only; Host departure does not end Table.
- Canonical WebSocket action path; WebRTC transient-only.
- Rejected actions consume Sequence; top-level action transactions are atomic.
- Rollback/replay with renderer reconciliation, not renderer rollback.
- Versioned deterministic PRNG; no Math.random gameplay.
- Lua is public creator language; persistent state Kernel-owned; guards read-only; no unrestricted browser/network access.
- One logical Room DO per active Table; DO sequences, clients simulate.
- Official hidden-info privacy but no malicious-client secrecy guarantee.
- Project/Runtime separation; command-based Editor; autosave; revision-protected single writer.
- Immutable Releases; Rooms and Saves pin exact Release.
- Processed immutable UGC assets; public UGC requires moderation/security gate.
- Host-only persistent Save creation in MVP.
- v1 prohibits nested live Prefab instances.

A.3 Baselines

- Bun monorepo/tooling
- Wasmoon Lua implementation
- D1 relational metadata
- R2 larger immutable/project/release/snapshot blobs
- Full-mesh WebRTC for 1-8 active players
- JSON wire protocol and canonical JSON snapshot encoding initially
- SHA-256 hashes
- GLB-first model import
- React FlexLayout, CodeMirror 6
- DO WebSocket hibernation where appropriate
- ~30s reconnect grace, ~30m empty-room expiry as configurable policies

A.4 Phase A open decisions

Wasmoon suitability
Lua instruction budget
PRNG algorithm
canonical numeric policy
transform precision
reference hardware
scene/asset budgets
snapshot budget
rollback strategy
checkpoint cadence
action retention
WebRTC topology
TURN
transient rate
mobile classification
final active-player cap
join-code format/entropy

A.5 Latest UX/DX decisions - v2

- LOCKED - semantic/simple object inspectors are the primary creator UX; raw components live under Advanced.
- LOCKED - v1 includes a small declarative Behavior Builder using authored AST -> deterministic generated Lua -> ordinary Lua runtime.
- LOCKED - no separate visual behavior runtime and no bidirectional Lua/visual round trip.
- LOCKED - Convert to Lua is one-way and undoable; generated diagnostics source-map to Behavior UI.
- LOCKED - templates are executable starter games.
- LOCKED - Lua standard library has its own pinned compatibility version; baseline modules turns, rounds, scores and teams.
- LOCKED - creator diagnostics are domain-aware and may offer undoable Quick Fixes.
- LOCKED - contextual Use in Script creates/inserts stable refs.
- LOCKED - no general live multiplayer Undo.
- LOCKED - sandbox Trash is canonical soft removal that retains EntityId; true destroy remains permanent.
- LOCKED - canonical game lifecycle INITIALIZING -> ACTIVE -> FINISHED; finished games block ordinary gameplay by default.
- DEFERRED - generic seamless Restart Game.
- LOCKED - creator-facing release vocabulary Draft / Test / Published.
- OUT - community remixing and community asset marketplace/packs in MVP.
- LOCKED - Test Online may coexist with continued Draft editing, but the running temporary Release remains immutable.
- LOCKED - Player errors use product-language recovery states; protocol/hash details remain creator/internal.

Appendix B - Component Registry v1

Status: Normative registry

Purpose: Define supported authored/runtime components, dependencies and creator-facing Add mappings.

Component	Core fields	Requires	Notes
transform	position, rotation, scale	-	Canonical values validated/quantized.
model	assetId, visible	transform	No Babylon handles in state.
collider	shape/config	transform	Interaction/presentation support.
grabbable	enabled; runtime heldBy	transform	Exclusive canonical holder.
lockable	locked	-	Blocks ordinary Player grab/move.
flippable	flipped	-	Semantic flip; renderer animates.
stackable	enabled	-	Allows Stack semantics.
card	definitionId, faceUp	transform	Definition data immutable.
container	items, capacity, ordering, visibility	-	Canonical membership store.
deck	Deck semantics	container	Top is last Container item.
hand	ownerSeat/player, canonicalOrder	container	Private official-client presentation.
die	definitionId, value	transform	Roll uses canonical RNG.
counter	value/default/min/max	-	set/add clamp to bounds.
zone	box/sphere, acceptedTags, visibleInPlay	transform	Semantic membership.
snap-point	radius/capacity/tags/alignment	transform	Nearest then stable ID tie-break.
text	value	-	Canonical when game logic changes it.
button	enabled, label	-	Canonical press.
script	scriptId, bindingId, props	-	Stable callback binding order.

B.1 Creator Add menu

Empty Object -> Transform

Card -> Transform + Card + Grabbable + Flippable + Stackable + Collider + presentation

Deck -> Transform + Container + Deck + Grabbable + Collider + presentation

WEB TABLETOP PLATFORM - FIRM HANDOFF PACKAGE

```
Die -> Transform + Die + Grabbable + Collider + presentation
Counter -> Counter + presentation
Container -> Container
Zone -> Transform + Zone
Snap Point -> Transform + SnapPoint
Hand -> Container + Hand
Text -> Text (+ Transform when in-world)
Button -> Button (+ Transform/presentation when in-world)
Model -> Transform + Model
```

B.2 Stack model

Stack is a specialized canonical Container-like record with stable StackId and ordered Entity IDs. v1 supports create, add, remove-top, merge and dissolve; arbitrary middle split is not required.

Appendix C - Canonical Action Registry v1

Status: Normative registry

Purpose: Define canonical action families, sources, prediction defaults and semantic constraints.

Action	Allowed source	Payload summary	Predict?	Key rule
system.game_start	system	settings	no	Initialize game and on_start
system.player_joined	system	playerId	no	Canonical lifecycle join
system.player_left	system	playerId	no	Voluntary logical departure
system.player_disconnect	system	playerId	no	Grace-expired departure; releases held objects
system.player_removed	system	playerId	no	Kick/admin removal
system.seat_assign	system	playerId, seatId	no	Canonical seat assignment
system.timer_fire	system	timerId	no	Exactly-once one-shot timer fire
entity.grab	player	entityId	yes	Requires free enabled grabbable and guard allow
entity.drop	player	entityId, transform	yes	Actor holds; quantize; resolve zones/snaps/stacks
entity.move	script/system	entityId, transform	no	Scripted canonical movement
entity.flip	player/script	entityId	yes	Requires flippable; Player guard
entity.set_locked	script (+ sandbox Player)	entityId, locked	maybe	Permission-gated
entity.spawn	script/system (+ sandbox Player)	prefabId, transform?, props?	no	Deterministic EntityId
entity.destroy	script/system (+ sandbox Player)	entityId	no	Cleans memberships/refs
container.move	script/internal	entityId/from/to/index	no	Atomic exclusive transfer
deck.shuffle	player/script	deckId	no	Canonical RNG; no resulting order payload
deck.draw_to_container	player/script	deckId,target,count	no	Card identity derived from Deck
deck.draw_to_world	player/script	deckId,transform?	yes-ish	Top card extracted to world
deck.deal	player/script	deckId,targets,cardsEach	no	Round-robin; insufficient cards reject
stack.remove_top	player/script	stackId	maybe	Identity from canonical top
die.roll	player/script	entityId	no	Kernel RNG chooses face
die.set_value	script	entityId,value	no	Value must match valid face
counter.set	script (+ permitted Player)	entityId,value	yes	Clamp
counter.add	script (+ permitted Player)	entityId,amount	yes	Clamp
button.press	player	entityId	yes	Enabled + can_press
text.set	script	entityId,value	no	Bounded string
snap.attach	script/internal	snapPointId,entityId	no	Capacity/compatibility
prompt.respond	player	promptId,response	yes	Must target Player and satisfy prompt schema

C.1 Prediction recommendation

Predict initially: entity.grab, entity.drop, entity.flip, button.press, simple counter interaction, prompt.respond

Do not predict initially: deck.shuffle, deck.deal, die.roll, entity.spawn/destroy, large scripted actions, timers

C.2 Derived events

```
game.started  
player.joined/left/disconnected/removed  
seat.assigned/left  
entity.spawned/destroyed/grabbed/dropped/flipped  
container.added/removed/moved  
deck.shuffled/drawn/dealt  
stack.created/changed/dissolved  
die.rolled  
counter.changed  
zone.entered/left  
snap.attached/detached  
button.pressed  
text.changed  
prompt.created/responded/canceled  
timer.fired  
action.rejected  
script.error
```

Appendix D - Reference Schemas, Glossary & Traceability

Status: Reference contract

Purpose: Show the intended serialized boundaries and common terminology.

D.1 Project example

```
{
  "schemaVersion": 1,
  "projectId": "project_space_traders",
  "name": "Space Traders",
  "defaultSceneId": "scene_main",
  "runtime": {"kernelVersion": 1, "luaApiVersion": 1},
  "playerSettings": {"minPlayers": 2, "maxPlayers": 6, "spectatorsAllowed": true},
  "interactionMode": "scripted",
  "scenes": ["scene_main"],
  "prefabs": ["prefab_pawn"],
  "scripts": ["script_game", "script_score_zone"],
  "assets": ["asset_board", "asset_card_back"],
  "scriptReferences": {"main_deck": "ent_main_deck", "score": "ent_score"}
}
```

D.2 Release manifest example

```
{
  "formatVersion": 1,
  "gameId": "game_space_traders",
  "releaseId": "release_space_traders_12",
  "releaseNumber": 12,
  "kernelVersion": 1,
  "luaApiVersion": 1,
  "networkProtocolVersion": 1,
  "defaultSceneId": "scene_main",
  "interactionMode": "scripted",
  "files": [{"path": "runtime/game.json", "contentHash": "sha256:...", "byteLength": 28142}],
  "integrity": {"manifestHash": "sha256:..."}
}
```

D.3 Snapshot example

```
{
  "formatVersion": 1,
  "kernelVersion": 1,
  "releaseId": "release_space_traders_12",
  "sequence": 1848,
  "state": {
    "settings": {"starting_gold": 10},
    "rng": {"algorithm": "TBD-v1", "state": ["..."], "draws": 42},
    "players": {}, "seats": {}, "entities": {}, "scriptState": {}, "prompts": {}
  },
  "stateHash": "sha256:..."
}
```

D.4 Wire examples

ActionRequest:

```
{"type": "action_request", "protocolVersion": 1, "requestId": "req_91", "predictedAtSequence": 1848, "action":
{"type": "entity.grab", "payload": {"entityId": "ent_red_pawn"}}
```

OrderedAction:

```
{"type": "ordered_action", "protocolVersion": 1, "sequence": 1849, "actionId": "act_room123_1849", "requestId": "req_
91", "actor": {"type": "player", "playerId": "player_alice"}, "action": {"type": "entity.grab", "payload":
{"entityId": "ent_red_pawn"}}
```

D.5 Glossary

Term	Definition
Account	Persistent authenticated platform identity.
Player	Logical participant in one Table; reconnect keeps same PlayerId.

WEB TABLETOP PLATFORM - FIRM HANDOFF PACKAGE

Connection	One active network connection for a Player.
Host	Room administrator; not simulation authority.
Project	Mutable creator-authored source.
Game	Stable public platform identity.
Release	Immutable playable build of a Game.
Table	User-facing live multiplayer session.
Room	Engineering/service object coordinating a Table.
Game Kernel	Pure deterministic semantic simulation runtime.
Canonical State	Gameplay state identical on healthy simulation clients.
Presentation State	Local visual/UI state that does not determine gameplay.
ActionRequest	Unsequenced Player request sent to Room service.
OrderedAction	Canonical sequenced input distributed to simulation clients.
Sequence	Monotonic position in one room canonical stream.
Prediction	Local speculative application before canonical ordering.
Rollback	Restore earlier state and replay to reconcile.
Snapshot	Serialized canonical state at one Sequence.
StateHash	Hash of canonical serialization.
Checkpoint	Snapshot accepted by room recovery infrastructure.
Entity	Stable logical game object.
Component	Composable semantic/configuration behavior on an Entity.
Container	Canonical holder/order of Entity IDs.
Prefab	Reusable authored Entity hierarchy template.
Stable Reference	Lua refs.* binding resolving by stable identity.
Script Binding	Stable association between script and Game/Entity runtime scope.
Transient Data	Networked presentation info such as cursor or drag preview.

D.6 Golden fixtures

Fixture	Proves
Card Deal	RNG, Deck shuffle/deal, Hands, snapshot/replay
Dice Race	Die RNG, Counter, scripted movement, SnapPoint
Private Hand	Hand contents, face state, official visibility projection
Conflicting Grab	Prediction, ordering, exclusive holder, rollback
Scripted Zone	Zone membership, Lua callback, Counter, Timer, transaction

Appendix E - Architecture Diagrams

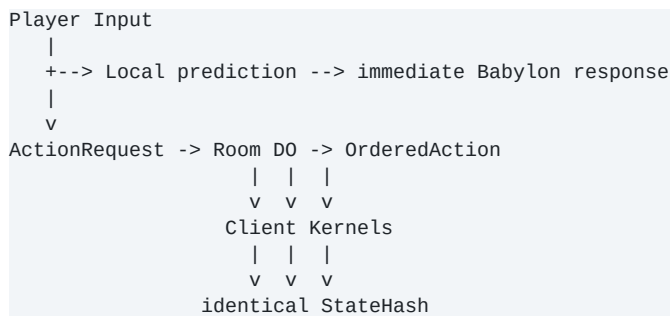
Status: Reference diagrams

Purpose: Communicate system boundaries and flows to technical and non-technical stakeholders.

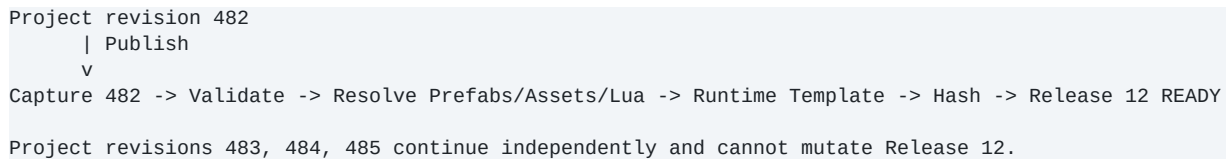
E.1 System context



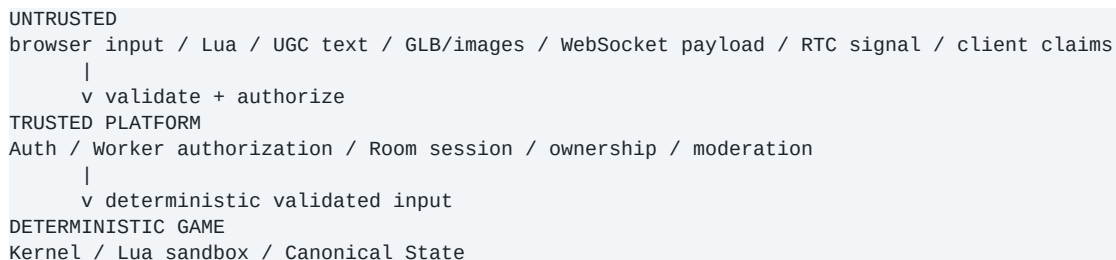
E.2 Canonical gameplay



E.3 Project to Release



E.4 Security boundaries



Appendix F - Vendor Response Template

Status: Procurement template

Purpose: Force vendors to respond on the technical risks that matter rather than generic marketing.

F.1 Required response areas

- Company/contact/team and actual engineer allocations.
- Up to five genuinely relevant projects.
- Top five architecture risks.
- Challenges to LOCKED decisions and first BASELINE validations.
- Cross-browser determinism strategy and PRNG recommendation.
- Lua sandbox, budgeting, memory and atomic failure implementation.
- DO sequencing, deduplication, rollback, checkpointing, reconnect and restart.
- WebRTC/TURN methodology.
- Babylon edit/runtime architecture and performance testing.
- Editor-core, undo, autosave, revisions, prefabs, CodeMirror and Playtest isolation.
- Asset/security threat handling.
- Testing strategy: unit, fuzz, simulation, E2E, performance, soak, chaos, security and usability.
- Phase A spike method/pass/fail criteria.
- Separate Phase A-G estimates, recurring external costs, exclusions, recommended scope reductions and stop/redesign conditions.

F.2 Mandatory scenario questions

Conflicting grab: Alice 180ms RTT, Bob 80ms RTT, both predict, Bob receives Sequence 501, Alice 502. Explain all meaningful client/server/Kernel states.

Lua atomicity: `score:add(1); deck:shuffle(); error("oops")`. What state remains and how is that guaranteed?

Compatibility: old Save pins Kernel v1/Lua API v1 while current Editor is newer. What runs?

Appendix G - Vendor Pricing Template

Phase	Contract model	Effort	Calendar	Price	Confidence
A Validation					
B Core Runtime					
C Creator + Local Play					
D Multiplayer					
E Publishing					
F UGC Readiness					
G Launch					

Vendor must separately list role rates/allocations, third-party recurring costs, assumption sensitivity (8->16 players, WebRTC removed, custom UI/nested Prefabs/voice/secure hidden-state added), change-order rates and optional stabilization/support.

Appendix H - Risk Register & Phase A Decision Gates

Status: Living risk register

Purpose: Separate intentional validation questions from missing product design.

ID	Risk	Prob.	Impact	Mitigation	Gate
RISK-001	Cross-runtime determinism	Medium	Critical	Canonical serializer, finite numbers, quantized geometry, explicit ordering/PRNG, fuzz fixtures	0 unexplained divergence across 10k+ seeded scenarios
RISK-002	Lua cannot be reliably bounded	Medium	Critical	Instruction hooks/instrumentation or replace runtime	Infinite loop/recursion/command explosion terminate safely
RISK-003	Lua sandbox escape	Low/Med	Critical	Adversarial sandbox tests	No arbitrary JS/browser/network capability
RISK-004	Rollback stalls	Medium	High	Compact state, local snapshots, efficient replay, conservative prediction	Benchmark 100/500/1000 action windows
RISK-005	Snapshot size excessive	Medium	High	Semantic-only state, compression, budgets, bounded Lua state	Benchmark 100-10k Entities and set limits
RISK-006	Babylon UGC performance	High	High	Processed assets, instancing/aggregation, creator warnings, hard budgets	Reference scenes meet approved targets
RISK-007	Mobile interaction confusing	Medium	High	Early touch prototype/usability	Choose phone+tablet/tablet-first/defer explicitly
RISK-008	Full-mesh WebRTC complexity	Medium	Medium	Canonical gameplay independent; fallback available	Choose full mesh/socket/other from tests
RISK-009	TURN cost/complexity	Medium	Medium	Representative NAT tests	Decide required/recommended/not needed
RISK-010	DO sequencing latency	Low/Med	High	Prediction, low tabletop action frequency, transient P2P	Measure regions/latency and UX
RISK-012	Hidden-info expectations exceed security	Medium	High	Accurate product wording and no competitive claims	Accepted residual risk documented
RISK-013	Editor scope explosion	High	High	Strict exclusions and phase change control	No visual scripting/collab/nested prefabs/custom UI before need
RISK-014	Lua API becomes awkward forever	Medium	High	Small semantic v1, versioning, real fixture games	API review before stable v1
RISK-015	Compatibility cost grows indefinitely	High	High	Version pinning + business support policy	Policy before broad public launch
RISK-016	UGC moderation burden	High	Med/High	Auth publishing, reports, moderation, no reviews initially	Phase F gate
RISK-017	Asset parser vulnerability	Low/Med	Critical	Minimal formats, limits, updated libs, malformed fixtures	Security review

WEB TABLETOP PLATFORM - FIRM HANDOFF PACKAGE

RISK-018	Autosave data loss	Medium	High	Revisions, write lease, local recovery buffer	Forced conflict/crash acceptance
RISK-019	Vendor builds demo architecture	Medium	Critical	Phased procurement, tests, source ownership, staging, traceability	Independent handover test
RISK-020	Scope/budget mismatch	High	High	Phase A, MUST/SHOULD/MAY, strict exclusions	Cut secondary polish before correctness boundaries

H.1 Mandatory Phase A decision outputs

A-DEC-001 Wasmoon
 A-DEC-002 Lua budget mechanism
 A-DEC-003 PRNG
 A-DEC-004 numeric policy
 A-DEC-005 transform precision
 A-DEC-006 reference hardware
 A-DEC-007 scene budgets
 A-DEC-008 snapshot budgets
 A-DEC-009 rollback strategy
 A-DEC-010 checkpoint cadence
 A-DEC-011 action retention
 A-DEC-012 WebRTC topology
 A-DEC-013 TURN
 A-DEC-014 transient rate
 A-DEC-015 mobile classification
 A-DEC-016 max active players

Appendix I - Vendor Scoring Matrix

Status: Procurement scoring

Purpose: Compare engineering competence and risk discipline independently of price.

Category	Weight
Architecture & deterministic systems	20
Realtime multiplayer	15
Browser 3D / Babylon	12
Lua / sandbox execution	12
Editor / complex frontend	10
Testing strategy	10
Product design / usability	7
Security & public UGC	5
Cloudflare / operations	4
Commercial structure	3
Communication quality	2

Each category scores 0-5 and is normalized by weight. A vendor may be rejected regardless of total for critical misunderstandings such as Host authority, syncing Babylon as canonical state, WebRTC-only authority, unrestricted Lua->JS, mutable Releases, client-trusted admin claims, or deferring deterministic/reconnect/security testing.

I.1 Shortlist technical exercises

Use the conflicting-grab scenario, Lua atomicity scenario and old-Release compatibility scenario as scored interviews. Strong vendors distinguish local prediction from canonical state, can explain transaction staging/commit, and separate Project migration from Release compatibility.

Appendix J - Phase A Acceptance Worksheet

Status: Validation sign-off

Purpose: Convert Phase A from exploratory research into explicit pass/conditional-pass/fail decisions.

Test	Method	Pass/Output
KERNEL-01 Cross-runtime determinism	10,000+ seeded scenarios across Chrome/Firefox/Safari/Bun	0 unexplained deterministic divergences
KERNEL-02 Rollback performance	100/500/1000-action windows across small/medium/large state	Approved ordinary window has no unacceptable stall
LUA-01 Wasmoon suitability	API surface, state, refs, timers, prompts, VM rebuild, diagnostics	Core locked requirements viable
LUA-02 Execution interruption	infinite loop, recursion, command explosion	Terminates safely without partial mutation
LUA-03 Memory containment	large tables/strings/state/module stress	Controlled failure and documented limits
LUA-04 Sandbox escape	window/document/fetch/WebSocket/storage/arbitrary JS/path traversal	No unrestricted capability
RNG-01 PRNG selection	reference vectors across runtimes	Exact algorithm/version/state locked
BAB-01 Tabletop performance	250 visible mixed objects + interpolation/hands	Approved reference target met
BAB-02 Deck presentation	multiple 52-card decks	No need for all contained cards as active meshes
BAB-03 Mobile/tablet	core touch tasks	Explicit support classification
NET-01 DO sequence continuity	active room through hibernation/restart	No reused/lost committed Sequence
NET-02 Request idempotency	duplicate normal/reconnect/restart	Exactly one OrderedAction
NET-03 Prediction/reconciliation	conflicts at 50/150/250ms+jitter	Immediate local response + final convergence
NET-04 Snapshot/reconnect	recent + stale reconnect	Catch-up/resync final hash matches
NET-05 WebRTC topology	2/4/6/8 players	Topology decision with measurements
NET-06 TURN	home/mobile/restrictive networks	Required/recommended/not-needed decision
DATA-01 Snapshot scale	100/1k/5k/10k Entities	Hard/recommended limits established
DATA-02 Checkpoint cadence	short/ordinary/6h room models	Cadence and retention locked
ASSET-01 GLB processing	ordinary/large/malformed/remote refs	Unsafe rejected; budgets established
EDIT-01 Project recovery	network loss/tab crash/takeover/stale revision	No silent overwrite; documented recovery

J.1 Full Phase A vertical slice

Creator: create simple project -> Deck + Die + Counter + Zone + Lua
 Local: playtest -> trigger script -> stop -> Project unchanged
 Multiplayer: second browser joins -> draw/move/roll/score
 Network: latency -> Host disconnect -> reconnect
 Final: all simulation clients have same canonical hash

J.2 Final outcome

GO - no blocking architecture issue
 CONDITIONAL GO - viable with specific owned conditions
 REDESIGN - foundational assumption failed; ADR/re-estimate required
 STOP - technical/business feasibility threshold not met

Appendix K - Bid Comparison Worksheet

Status: Selection worksheet
Purpose: Normalize scope and compare technical quality, estimate realism and price.

Vendor	Technical /100	Phase A price	Later estimate	Confidence	Recommendation
Vendor A					
Vendor B					
Vendor C					

K.1 Compare risk positions

Topic	Vendor A	Vendor B	Vendor C
Deterministic architecture			
Wasmoon confidence			
Lua budget approach			
PRNG recommendation			
DO sequence/restart			
Prediction/rollback			
WebRTC/TURN			
Babylon performance			
Editor architecture			
UGC security			

K.2 Estimate realism

Classify each estimate as Aggressively optimistic, Optimistic, Credible, Conservative or Unclear. Normalize exclusions before comparing totals. Phase A vendor selection should maximize technical understanding, risk discovery, prototype discipline, testing rigor and communication rather than lowest production quote.

Final Handoff Status

FIRM-READY FOR PHASE A SOLICITATION

The product, architecture, UX, deterministic simulation, Lua API, multiplayer protocol, Editor system, platform/security model, component/action vocabularies, acceptance gates, vendor SOW, scoring and risk framework are sufficiently defined for multiple competent firms to bid against the same project and be evaluated objectively.

The remaining unresolved decisions are deliberate Phase A validation questions rather than missing specification: Wasmoon execution control, exact PRNG, numeric/quantization policy, snapshot/rollback budgets, WebRTC/TURN topology, hard scene/asset limits, mobile support classification, active-player cap and final join-code entropy/format.

Final package structure

SPEC 00-11 + Appendices A-K. This revision supersedes the prior artifact and includes the finished UX/DX contract, screen-by-screen flows, Behavior AST compiler design, Lua stdlib versioning, semantic inspectors, domain-aware diagnostics, and sandbox Trash semantics.

- SPEC 00 - Executive Brief
- SPEC 01 - Product Requirements
- SPEC 02 - UX & Interaction Specification
- SPEC 03 - Deterministic Game Kernel
- SPEC 04 - Lua API v1
- SPEC 05 - Multiplayer Protocol & Room Service
- SPEC 06 - Editor & Creator System
- SPEC 07 - Platform, Data, Security & UGC
- SPEC 08 - Acceptance, Delivery & Traceability
- SPEC 09 - Statement of Work & Vendor Procurement Requirements
- SPEC 10 - Finished UX & Creator DX Contract
- SPEC 11 - Screen-by-Screen UX & DX Contract
 - Appendix A - Consolidated Decision Register
 - Appendix B - Component Registry v1
 - Appendix C - Canonical Action Registry v1
 - Appendix D - Reference Schemas, Glossary & Traceability
 - Appendix E - Architecture Diagrams
 - Appendix F - Vendor Response Template
 - Appendix G - Vendor Pricing Template
 - Appendix H - Risk Register & Phase A Decision Gates
 - Appendix I - Vendor Scoring Matrix
 - Appendix J - Phase A Acceptance Worksheet
 - Appendix K - Bid Comparison Worksheet

This PDF intentionally consolidates and supersedes exploratory discussion. If a future decision changes a LOCKED item, the change should be recorded through the ADR and requirements change-control process defined in this package.